IEEE INFOCOM 2025 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS) | 979-8-3315-4370-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/INFOCOMWKSHPS65812.2025.11152765

The Merit of Simple Policies: Buying Performance With Parallelism and System Architecture

Mert Yildiz, Alexey Rolich, Andrea Baiocchi
Dept. of Information Engineering, Electronics and Telecommunications (DIET), University of Rome Sapienza, Italy
mert.yildiz@uniroma1.it alexey.rolich@uniroma1.it andrea.baiocchi@uniroma1.it

***Abstract*—While scheduling and dispatching of computational workloads is a well-investigated subject, only recently has Google provided publicly a vast high-resolution measurement dataset of its cloud workloads. We revisit dispatching and scheduling algorithms fed by traffic workloads derived from those measurements. The main finding is that mean job response time attains a minimum as the number of servers of the computing cluster is varied, under the constraint that the overall computational budget is kept constant. Moreover, simple policies, such as Join Idle Queue, appear to attain the same performance as more complex, size-based policies for suitably high degrees of parallelism. Further, better performance, definitely outperforming size-based dispatching policies, is obtained by using multi-stage server clusters, even using very simple policies such as Round Robin. The takeaway is that parallelism and architecture of computing systems might be powerful knobs to control performance, even more than policies, under realistic workload traffic.**

***Index Terms*—Data centers; Scheduling; Dispatching; Large-Scale multi-server system; Workload traffic measurements;**

I. Introduction

The increasing demand for data-driven applications and the complexity of distributed computing environments have made job scheduling and dispatching in large-scale cluster systems a critical area of study. These clusters, which support cloud computing and high-performance computing infrastructures, need to balance system utilization, minimize response times, and ensure fair resource distribution. The challenge is further complicated by the variability of workloads, which range from short, bursty tasks to long, resource-intensive jobs.

Enhancing the performance of large-scale clusters is closely tied to the efficiency of the dispatching and scheduling algorithms employed. Dispatching consists of assigning tasks to servers, while scheduling determines the sequence in which tasks are executed within each server. Both processes influence key performance metrics such as mean response time, system throughput, and load balancing. Traditional dispatching algorithms like Round Robin (RR) and scheduling algorithms like First Come First Served (FCFS) are simple, but often ineffective in environments with heterogeneous tasks, where job sizes and resource requirements can vary significantly.

This work was partially supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, in partnership with Telecommunications of the Future (PE0000001 - program "RESTART").

To overcome these limitations, advanced dispatching strategies such as Join Shortest Queue (JSQ), Join Below Threshold (JBT), Size Interval Task Assignment (SITA) and Least Work Left (LWL) , along with scheduling algorithms like Shortest Remaining Processing Time (SRPT), have been introduced to enhance load balancing and reduce job completion times. It is known that in the case of non size-aware policies JSQ is heavy-traffic delay-optimal [1, Ch. 10], as well as JBT [2]. As for size-based dispatching policies, it is known that there is no definite winner between SITA and LWL [3], yet they are both superior to other dispatching policies under the assumption of FCFS servers. However, under heavy traffic, authors in [4] propose the first size- and state-aware dispatching policy called CARD (Controlled Asymmetry Reduces Delay), which keeps all but one of the queues short, then routes as few jobs as possible to the one long queue that provably minimizes mean delay in heavy traffic. Finally, it is well known that SRPT minimizes the job response time in a single server system, within the class of size-based, pre-emptive policies. Moreover, multi-stage systems offer a promising solution for optimizing resource allocation by categorizing tasks based on their service times, improving performance in dynamic and diverse environments. However, the success of these approaches largely depends on the characteristics of the workloads and system configurations. In real-world scenarios, for example, the service times of incoming tasks are often unknown, which makes size-based policies hard to implement.

This paper examines the performance of various dispatching and scheduling algorithms in large-scale cluster systems, leveraging real-world data from Google's Borg clusters [5], [6]. We assess how these algorithms impact response times and system slowdowns across different load conditions and analyze the influence of parallelism on performance. Additionally, we propose a novel two-stage dispatching system tailored for managing highly diverse workloads, achieving notable efficiency gains compared to traditional single-stage methods.

This research contributes to the ongoing efforts to optimize resource allocation in large-scale distributed systems, offering valuable insights for improving the performance of cloud computing and high-performance clusters. More in depth, the specific contributions of this paper are as follows:

- We assess the role of parallel servers, by keeping the overall computational power budget fixed, showing that simple, suboptimal policies can achieve as good perfor-

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:11:32 UTC from IEEE Xplore. Restrictions apply.

mance as known optimal ones (*break down correlations and improve performance by means of parallel servers rather than complex policies*).
- We give evidence that a suitable system architecture (two-stage server system) can outperform single-stage one, in spite of the latter using much more powerful policies than the former system (*buy performance gain with design of system architecture, rather than complex policies*).

The remainder of this paper is organized as follows: Section II reviews related work on dispatching and scheduling algorithms for large-scale systems. Section III details the dataset and key metrics used in our simulations. In Section IV, we introduce our proposed approach. Section V describes our experimental setup, while Section VI analyzes the simulation results. Finally, Section VII summarizes our findings and discusses potential directions for future research.

II. Background and Related Work

In recent years, the management of resources in data centers has been extensively studied from multiple perspectives, particularly in terms of determining the optimal amount of computational power and effectively allocating resources to various jobs. Researchers have explored various strategies to balance energy efficiency, performance, and scalability, focusing on both the hardware and software layers.

Optimizing dispatching and scheduling in multi-server environments remains a complex task, with no universally optimal solution identified [7]. While significant advances have been made in the analysis and optimization of single-server systems, the multi-server context presents challenges such as heterogeneous resource demand, and load distribution across servers, making it difficult to apply a single solution across different scenarios [8].

The optimality of a dispatching policy heavily depends on the information available to the dispatcher. When job sizes are unknown but server states (e.g., queue lengths or workloads) are accessible, policies like RR [9], [10], JSQ [11], and LWL [12] are proven to be optimal by directing jobs to the least-loaded servers. On the other hand, when only job sizes and their distribution are known, the SITA policy is considered optimal [13]. However, these assumptions often do not align with the complexities of real-world workloads.

In [14] and [15], the authors make a strong case for enhancing energy efficiency by utilizing fewer, more powerful servers. While traditional dispatching and scheduling methods often indicate that a higher number of servers can reduce mean response time, our innovative two-stage approach shows superior performance even with fewer servers that have higher computational power. By applying standard algorithms across both stages, the system effectively optimizes workload distribution and resource management. This results in better performance compared to conventional methods. Furthermore, consolidating tasks onto fewer powerful servers reduces both energy consumption and operational overhead, as these servers can efficiently handle more complex workloads, thereby avoiding the inefficiencies associated with managing a larger fleet of lower-powered servers.

A large heterogeneous server cluster is considered in [16] by means of a discrete-time (slotted) model, assuming Bernoulli arrivals and Geometric service times. Under limited knowledge of server capacity and with delayed queue length information, the authors argue that there is no point in resorting to complex policies such as JSQ, whereas weighted RR grabs most of the potential performance. The emphasis of this work is on the optimization of mean system response time under limited knowledge of server capacity, with a bandit-based exploration.

In our work, we explore the effectiveness of large-scale cluster sizes in taming real-world workload variability. Inspired by [14], [15], we then consider a two-stage architecture for the server cluster, to reap as good performance as with large-scale systems, while not requiring a very large number of servers. Also, following [16], but using real-world workload measurement, we show that RR is still competitive with respective to more sophisticated policies. Instead of using dynamic learning systems, we show that a substantial improvement in performance is achievable by playing with system architecture.

In our investigation, we found that while the heuristic algorithm we tested may not be globally optimal, it outperformed some of the best-known policies in specific conditions. This observation aligns with recent findings, such as those discussed in [17], which highlight the difficulties of achieving optimal scheduling in complex, heterogeneous systems. Additionally, [18] explored optimal scheduling under heavy traffic in multi-server environments and emphasized the challenge of balancing workloads across servers while maintaining efficiency. Their findings further support the notion that, while heuristic methods may provide substantial improvements, achieving a universally optimal scheduling strategy remains elusive.

Our research addresses the role of server cluster size and architecture under real large-scale workload data, demonstrating that a two-stage system design can significantly enhance performance, even when using simpler, suboptimal policies. This approach emphasizes that optimizing system architecture, rather than relying solely on complex scheduling policies, can achieve superior results, providing valuable insights into the effective allocation of resources in large-scale distributed environments.

III. Workload Dataset Description

The section describes the Google comprehensive traffic dataset released in 2020, which captures user and developer activity measured over one month in 2019. This dataset provides detailed data on users' requests of Google's computational resources, such as Central Processing Unit (CPU), across eight data centers located in America, Europe, and Asia, referred to by Google as "Borg cells." Traffic measurements from each Borg cell consist of five tables with information about users' resource requests, the machines processing these requests, and the time evolution of each request through its processing states in Borg's scheduler.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:11:32 UTC from IEEE Xplore. Restrictions apply.

Users submit jobs, which may consist of one or more tasks, known as instances. Each instance requires a specific amount of CPU time to execute successfully. The dataset provides these quantities in units based on the so called Google Normalized Computing Unit (GNCU). The GNCU is the computational power of a default machine. CPU times provided for each task assume the server computational power equals 1 GNCU.

The life cycle and event types associated with jobs and tasks are crucial to understanding their execution. After a job is submitted, its tasks may either be processed immediately or placed in a queue, depending on the system's load. Upon completion, tasks are categorized based on their outcome. Google classifies successfully completed tasks as "FINISH". Those that do not complete successfully are labeled as "KILL," "LOST," or "FAIL," depending on the specific reason for the failure. Further details can be found in the documentation [19].

After conducting a detailed analysis, we extracted the key information necessary for managing resources in large-scale clusters [20]. Our focus was limited to tasks that were successfully completed ("FINISH" tasks), as incomplete tasks do not provide reliable insights into the resources required for completion.

Google's original dataset includes essential details for each task, such as job and task identifiers, the amount of CPU time utilized (averaged over 300-second intervals), and randomly sampled 1-second snapshots of CPU usage within each time window. Using this data, we reconstructed a workload description with the following features for each task:[1]

- A_{ij}: The arrival time of task i belonging to job j.
- C_{ij}: The CPU time required to execute task i of job j on a standard processor with a computational capacity of 1 GNCU, representing the service time for the task.

Google's dataset provides performance metrics for these "standard" processors, enabling consistent workload analysis. In this study, we focus on a simplified task representation, considering only the arrival times and the required CPU time for each task. This abstraction allows for a more streamlined evaluation of system performance while maintaining sufficient detail for meaningful insights.

IV. Framework for the Two-Stage Dispatching and Scheduling System

The two-stage dispatching system is designed to address the challenges of managing highly heterogeneous workloads in large-scale multi-server environments. The system separates tasks based on their service time, ensuring efficient resource utilization by isolating short tasks ("mice") from long tasks ("elephants"). This architectural approach aims to improve performance without relying on complex dispatching policies.

The layout of the two-stage system is illustrated in Figure 1. It consists of two clusters of servers: the first-stage cluster with N_1 servers and the second-stage cluster with $N_2 = N - N_1$ servers, where N is the total number of servers. Tasks arriving at the system are dispatched to the first-stage servers. If a task is completed within a predefined threshold θ, it exits the system. Otherwise, it is transferred to the second-stage cluster for further processing. The operation of the system is formalized in Algorithm 1.

Algorithm 1 Two-stage Dispatching and Scheduling System

Input: Task set $T = \{t_1, t_2, \ldots, t_n\}$, Threshold θ, Stage_1 Servers N_1, Stage_2 Servers N_2
Initialize: RR dispatchers and FCFS scheduling for servers for Stage_1 and Stage_2
procedure STAGE_1 DISPATCHING AND PROCESSING
  for each arriving task $t_i \in T$ **do**
    Dispatch t_i to Stage_1 server N_1 using RR
    while t_i is not completed **do**
      Process t_i on N_1 using FCFS
      if t_i is completed before θ **then**
        Exit system
      else
        Transfer t_i to Stage_2 dispatcher
        STAGE_2 PROCESSING(t_i)
      end if
    end while
  end for
end procedure
procedure STAGE_2 PROCESSING(t_i)
  Dispatch t_i to Stage_2 server N_2 using RR
  while t_i is not completed **do**
    Process t_i on N_2 using FCFS
  end while
  Exit system
end procedure

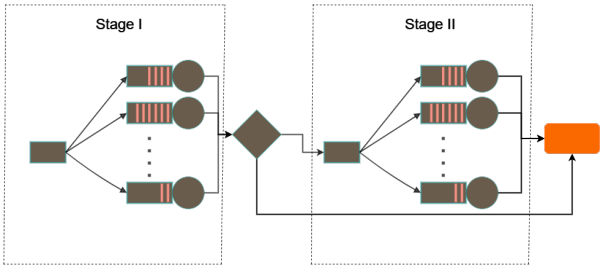


Figure 1. Layout of the Two-stage Multi-Server System.

This system design employs simple dispatching and scheduling policies to minimize implementation complexity. RR is used for dispatching, and FCFS is employed for scheduling at both stages. Despite its simplicity, the two-stage architecture effectively manages heterogeneous workloads through task separation and resource specialization.

The proposed framework emphasizes architectural design over sophisticated policies. By demonstrating the effectiveness of this design, the system showcases a practical and scalable solution for real-world distributed environments.

[1]The dataset used in this study, encompassing data from all eight Google data centers, is publicly available at https://github.com/MertYILDIZ19/Google_cluster_usage_traces_v3_all_cells.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:11:32 UTC from IEEE Xplore. Restrictions apply.

V. Simulation Setup

This section describes the simulation setup for evaluating multi-server systems using Google's workload dataset. Simulations are run by feeding the multi-server system with one day worth of workload.

Considered performance metrics are:

1) Mean Response Time (MRT), R: the average across all jobs of the time elapsing since when the first task of the job arrives, until when the last task of the job is completed.
2) Mean Job Slowdown (MJS), S: the average across all jobs of the ratio between the job response time and the maximum among service times required by all tasks belonging to the job.

Two different scenarios are defined for simulations: the single-stage service system in Section V-A, and the two-stage one in Section V-B.

A. Single-stage multi-server system

The workload built out of Google's data is fed to a multi-server system, comprising N parallel, fully accessible and equivalent servers. Each server is configured with μ GNCUs and adopts FCFS policy. Hence, the service time of task i of job j is given by $X_{ij} = C_{ij}/\mu$. The overall average utilization coefficient of the multi-server system (briefly referred to as load) is

$$\rho = \frac{\lambda \overline{X}}{N} \tag{1}$$

where λ is the mean task arrival rate computed as

$$\lambda = \frac{\max_{i,j} A_{ij} - \min_{i,j} A_{ij}}{\sum_{j=1}^{J} K_j} \tag{2}$$

and $\overline{X}$ is the average service time, given by

$$\overline{X} = \frac{\sum_{j=1}^{J}\sum_{i=1}^{K_j} X_{ij}}{\sum_{j=1}^{J} K_j} = \frac{1}{\mu}\frac{\sum_{j=1}^{J}\sum_{i=1}^{K_j} C_{ij}}{\sum_{j=1}^{J} K_j} = \frac{\overline{C}}{\mu} \tag{3}$$

Note that λ and $\overline{C}$ (the average service time when servers are endowed with one GNCU) are fixed numbers, once the Google dataset workload trace is given. The equations above imply that

$$\rho = \frac{\lambda \overline{C}}{\mu N} \tag{4}$$

Then, we define two kinds of analysis:

1) we vary the load ρ on the system by scaling μ, for a fixed number of servers N_0.
2) we vary the number of servers N by scaling μ, for a fixed load ρ_0.

Dispatching across servers is managed using one of several possible policies: RR, Join Idle Queue (JIQ), and LWL. These policies have been chosen so as to compare performance achieved by increasingly more complex policies: from the simple stateless RR to the stateful, but simple pull policy JIQ, up to the anticipative policy LWL.

Table I
Simulation parameters

Parameter	Value/Description
Number of Servers, N	[2, 3000]
Ratio of number of servers in two-stage multi-server system, N_1/N_2	1
Server Processing Rate, μ	Computed based on fixed overall capacity
System load, ρ	[0.5; 0.9]
Target fixed load, when varying N	0.6
Threshold, θ	{0.5, 1, 2, 5, 10, 20, 50, 75, 100, 200} sec
Dispatching Policies	RR, JIQ, LWL
Scheduling Policy	FCFS
Performance Metrics	Mean Response Time, Slowdown

B. Two-stage multi-server system

The two-stage system simulation leverages real-world workload traces from Google's Borg clusters to evaluate performance under diverse conditions. The system consists of N servers, each with computational power μ GNCU, divided equally between two stages: $N_1 = N_2 = N/2$. The total computational budget is maintained constant at $N\mu = \lambda\overline{C}/\rho_0$, where the system load ρ is fixed at ρ_0, and μ is adjusted to satisfy $\lambda\overline{C}/(\mu N) = \rho_0$.

Tasks are dispatched to the first stage using the round-robin (RR) policy. If a task is completed within a predefined threshold θ, it exits the system. Otherwise, it is transferred to the second stage for further processing, where both stages employ first-come, first-served (FCFS) scheduling. Different values of the threshold θ are tested, as outlined in Table I.

The performance metrics used for evaluating the two-stage system are consistent with those applied in the single-stage system, ensuring comparability. This design facilitates an analysis of the effects of task separation and resource specialization on system performance.

By varying the number of servers N and computational capacity μ, this simulation investigates the scalability and efficiency of the two-stage system. The setup provides insights into how parallelization and architectural design influence performance in heterogeneous workload scenarios.

VI. Simulation Results

This section presents the simulation results, according to the system set-up defined in Section V. We evaluate how dispatching policies, system architecture, and parallelism affect performance metrics like mean job response time and mean job slowdown, highlighting the benefits of the two-stage system over single-stage configurations.

Table I summarizes the key parameters used in our simulations. Notably, in our experiments, the number of servers in each stage of the two-stage system was kept the same for both stages. By maintaining a constant overall capacity while varying the number of servers (N) and the processing rate per server (μ), under the constraint that the product $N\mu$ be fixed, according to the fixed utilization coefficient ρ_0, we systematically examined how changes in parallelism and architectural design influence system performance.

These parameters were chosen to reflect realistic conditions often found in large-scale computing environments. For

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:11:32 UTC from IEEE Xplore. Restrictions apply.

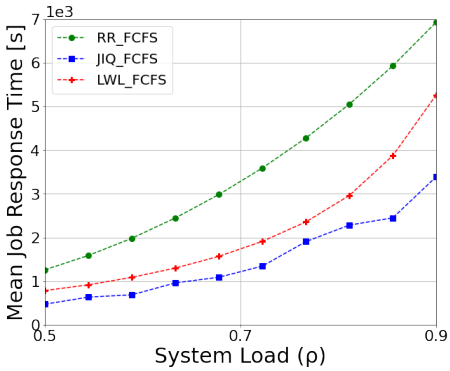


Figure 2. MRT as a function of system load using different job dispatching and scheduling algorithms.

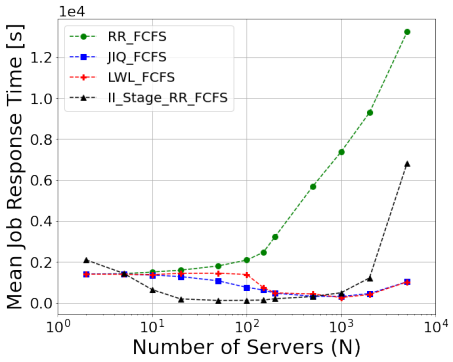


Figure 3. MRT as a function of the number of servers, demonstrating the performance impact of increasing parallelism.

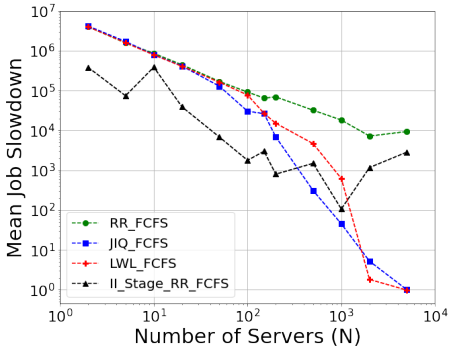


Figure 4. Job Slowdown as a function of the number of servers, illustrating the scalability benefits of parallelism.

instance, varying the number of servers allows us to assess the scalability of different dispatching policies under increasing parallelism while adjusting the processing rate per server provides insights into the trade-offs between server capacity and load distribution.

Performance metrics defined in Section V are used to quantify the effectiveness of each configuration, enabling a direct comparison between single-stage and two-stage systems. Our evaluation focused on three dispatching algorithms, namely RR, JIQ, and LWL, combined with FCFS scheduling within all servers. The simulation results highlight the critical impact of parallelism and dispatching strategies on system performance.

Figure 2 shows how MRT scales with system utilization coefficient ρ for different dispatching policies. RR performs poorly, especially under high loads, not surprisingly, given that it is the simplest dispatching policy. It does not account for the state of the server, nor does it assume knowledge of task sizes. As a consequence, MRT yields strongly worse MRT values than JIQ and LWL.

In contrast, more sophisticated strategies such as JIQ and LWL effectively leverage parallelism to balance the load across servers. JIQ assigns tasks to idle servers, ensuring workload distribution without requiring prior knowledge of task sizes. However, under heavy load with a fixed number of servers, JIQ is known to be suboptimal.

Figure 3 illustrates the MRT as a function of the number of servers for a given mean load $\rho_0 = 0.6$. As expected, RR performs the worst, showing no benefit from increased server parallelism. Instead, the MRT for RR increases with the number of servers N.

A more nuanced behavior is observed with JIQ and LWL. Initially, the MRT remains nearly constant. For moderate values of N (in the range of hundreds), the MRT decreases, benefiting from improved parallelism. However, as N grows beyond several thousand, the MRT starts to increase again.

This rise in MRT for large N is due to the inverse relationship between the number of servers and their individual processing rate μ, under the constraint $N\mu$. As N increases, μ decreases, resulting in slower processing speeds per server. While additional servers increase the likelihood of finding an idle server, the reduced speed of each server causes tasks to be processed more slowly overall. Consequently, the MRT, which is inversely proportional to μ, eventually grows unbounded as N approaches infinity.

The local minimum of MRT represents the optimal trade-off between server speed and parallelism. At this point, the benefits of reducing workload correlations through parallelism balance out the negative impact of slower server speeds. Beyond this point, the diminishing returns of parallelism and the effect of decreasing μ dominate, leading to an overall increase in MRT.

Another interesting point is that JIQ tends to achieve the same performance as LWL, even though the latter is exploiting the knowledge of task sizes and of server backlogs. Such detailed information, often hard to get in real systems, appear to be superfluous for large-scale cluster sizes.

Figure 4 illustrates the behavior of average job slowdown as the number of servers grows. With increasing server counts, all dispatching algorithms show improvements in performance. These results confirm the inherent strengths of parallelism.

Our proposed two-stage system stands out as the top performer, outperforming even the most sophisticated dispatching policies while employing the simplest RR strategy. As illustrated in Figure 3, the two-stage system consistently achieves lower MRT across different thresholds and server configurations. Remarkably, it delivers better MRT with just 20 servers than a single-stage system with 1000 servers, even when using the best dispatching policy among those

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:11:32 UTC from IEEE Xplore. Restrictions apply.

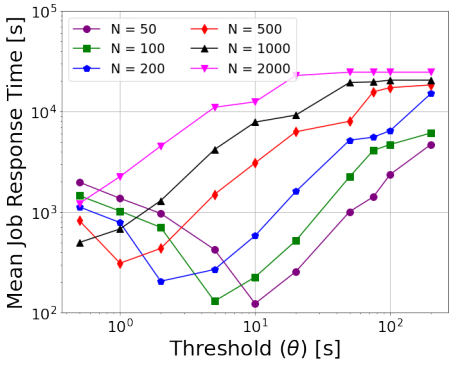


Figure 5. MRT of jobs for different server counts and threshold values in the two-stage system, showcasing its superior performance.

considered here. By isolating short tasks ("mice") from longer ones ("elephants"), the two-stage system minimizes delays for smaller tasks, leading to more efficient resource utilization and significantly improved response times. However, when the number of servers is relatively large, the two-stage system loses again single stage one systems. This is because the processing speed of each server is very low, causing tasks to get stuck in the first stage, while subtracting server to the second-stage to deal with bigger tasks. The two-stage system's performance is particularly noteworthy given its use of the RR dispatching policy, which is typically the least effective in single-stage scenarios. Despite this, our approach outperforms LWL with FCFS, demonstrating the profound impact of architectural design on system efficiency.

Figure 5 plots the MRT as a function of the service time threshold of the first stage, θ, for several values of N and a fixed value of the average utilization factor ρ_0. The optimal threshold value that minimizes the MRT is identified from these curves.

VII. Conclusion

In this paper, we evaluated the performance of various dispatching and scheduling algorithms in large-scale cluster systems using real-world workload measurements from Google's Borg clusters. Our proposed two-stage dispatching and scheduling system showed significant improvements in managing heterogeneous workloads, outperforming traditional single-stage models, including those employing powerful dispatching strategies like LWL. Overall, these results emphasize the importance of parallelism and architectural design of the server cluster system. Playing with those levers, it is possible to achieve as good performance (or even better than) those achievable with complex dispatching policies.

For future work, we plan to extend our approach by exploring systematic optimization of the parameters of the two-stage system (e.g., the threshold θ, the size of the first stage N_1, and possibly the capacity of servers in the first and second stage) is another interesting line of development. Finally, trying to gain more insight into system architecture optimization employing, possibly approximate, models is also worth of being pursued. It is especially interesting to understand up to what point simplifying assumptions on workload statistics, required to define models amenable to analysis, might lose key effects brought about by real-world workload traffic traces provided by Google's measurements.

References

[1] R. Srikant and L. Ying, *Communication Networks – An Optimization, Control, and Stochastic Networks Perspective*. Cambridge, UK: Cambridge University Press, 2014.

[2] X. Zhou, F. Wu, J. Tan, Y. Sun, and N. Shroff, "Designing low-complexity heavy-traffic delay-optimal load balancing schemes:theory to algorithms," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, dec 2017.

[3] M. Harchol-Balter, A. Scheller-Wolf, and A. R. Young, "Surprising results on task assignment in server farms with high-variability workloads," *SIGMETRICS Perform. Eval. Rev.*, vol. 37, p. 287?298, jun 2009.

[4] R. Xie, I. Grosof, and Z. Scully, "Heavy-Traffic Optimal Size- and State-Aware Dispatching," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 8, no. 1, pp. 1–36, 2024.

[5] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes, "Borg: the next generation," in *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, (New York, NY, USA), Association for Computing Machinery, 2020.

[6] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at google with borg," in *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15, (New York, NY, USA), Association for Computing Machinery, 2015.

[7] A. Chraibi, S. Ben Alla, A. Touhafi, *et al.*, "A novel dynamic multi-objective task scheduling optimization based on dueling dqn and per," *The Journal of Supercomputing*, vol. 79, pp. 21368–21423, 2023.

[8] M. Harchol-Balter, "Open problems in queueing theory inspired by datacenter computing," *Queueing Syst. Theory Appl.*, vol. 97, p. 3?37, feb 2021.

[9] Z. Liu and D. Towsley, "Optimality of the round-robin routing policy," *Journal of Applied Probability*, vol. 31, no. 2, pp. 466–475, 1994.

[10] Z. Liu and R. Righter, "Optimal load balancing on distributed homogeneous unreliable processors," *Operations Research*, vol. 46, no. 4, pp. 563–573, 1998.

[11] R. R. Weber, "On the optimal assignment of customers to parallel servers," *Journal of Applied Probability*, vol. 15, no. 2, pp. 406–413, 1978.

[12] O. T. AKGUN, R. RIGHTER, and R. WOLFF, "Partial flexibility in routeing and scheduling," *Advances in Applied Probability*, vol. 45, no. 3, pp. 673–691, 2013.

[13] H. Feng, V. Misra, and D. Rubenstein, "Optimal state-free, size-aware dispatching for heterogeneous m/g/-type systems," *Performance Evaluation*, vol. 62, no. 1, pp. 475–492, 2005. Performance 2005.

[14] İ. Çağlar and D. T. Altılar, "Look-ahead energy efficient vm allocation approach for data centers," *Journal of Cloud Computing*, vol. 11, no. 1, p. 11, 2022.

[15] S. Singh, I. Chana, J. Singh, and A. Agrawal, "A survey and taxonomy on energy management schemes in cloud data centers," *The Journal of Supercomputing*, vol. 78, no. 3, pp. 3533–3595, 2022.

[16] T. Choudhury, G. Joshi, W. Wang, and S. Shakkottai, "Job dispatching policies for queueing systems with unknown service rates," in *Proceedings of the Twenty-Second International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, MobiHoc '21, (New York, NY, USA), p. 181?190, Association for Computing Machinery, 2021.

[17] D. Efrosinin, V. Vishnevsky, and N. Stepanova, "Optimal scheduling in general multi-queue system by combining simulation and neural network techniques," *Sensors*, vol. 23, no. 12, 2023.

[18] I. Grosof, Z. Scully, M. Harchol-Balter, and A. Scheller-Wolf, "Optimal scheduling in the multiserver-job model under heavy traffic," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 6, no. 3, pp. 1–32, 2022.

[19] J. Wilkes, "cluster-data/Clusterdata2019." https://github.com/google/cluster-data/tree/master, 2019.

[20] M. Yildiz and A. Baiocchi, "Data-driven workload generation based on google data center measurements," in *2024 IEEE 25th International Conference on High Performance Switching and Routing (HPSR)*, pp. 143–148, 2024.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:11:32 UTC from IEEE Xplore. Restrictions apply.